Aleksandar Milicevic

Ph.D., Computer Science, MIT (2015)

CONTACT INFORMATION

e-mail: aca.milicevic@gmail.com
web: https://aleksandarmilicevic.github.io

SUMMARY

Security-focused systems engineer and technical leader with a Ph.D. in Computer Science from MIT, a strong background in formal methods (research in declarative programming, specification languages, software verification, and program synthesis), and over a decade of experience building and operating production infrastructure. At Cubist, founding engineer of a key-management platform securing over $2B in digital assets: built its policy engine and type-enforced security checks, and led the customer integrations. At Microsoft, tech lead for Linux in a distributed build-acceleration service, taking it from a two-week prototype to a technology adopted by about eight internal teams.

Managed two engineering teams at Cubist and mentored engineers into owners of major platform components.

EXPERIENCE

Cubist Inc
Founding Engineer **September, 2022 - October, 2026**

Founding engineer of a hardware-backed, non-custodial platform for programmatically managing cryptographic keys, built for high throughput and horizontal scalability. Keys are only ever handled inside AWS Nitro Enclaves, so no key material is exposed outside a small, verifiable boundary. Secures over $2B of staked BTC, serves customers including Ava Labs and ConsenSys, and runs at 99.99% availability. Cut one trading firm's signing latency from 600 ms to 24 ms.

Main contributor to the design and implementation of a *declarative policy engine*. Users write policies stating the conditions under which their keys may be used, such as spending limits, address allowlists, time windows, and multi-party approval. Every signing request is checked against these policies, with low latency on the request path. The same mechanism enforces MFA on any API route, such as login, organization settings, and policy edits. The policy engine passed an independent security audit by Veridise.

Co-designed the platform's end-to-end architecture, from the enclave-based signing path to the surrounding cloud services, and built the live monitoring and alerting that keeps it running around the clock, including automated triggers that open on-call incidents and surface production issues before they reach customers.

Played a central role in securing the system as a whole. Designed a family of Rust *guard types* that wrap sensitive data and grant access only once every required check has passed: for example, a signing key cannot be used until all applicable policies have been evaluated. This turns security invariants into compile-time guarantees, so a missed check is a type error rather than a vulnerability.

Led technical integration for key enterprise customers, including Ava Labs and ConsenSys: main technical contact, gathered requirements, built the features they needed (e.g., SSO login, configurable MFA), and guided them through onboarding. Delivered multi-region support in two months to close a latency-sensitive deal, using per-key region affinity to preserve single-region consistency guarantees.

Tech: AWS Nitro Enclaves, DynamoDB, Lambda, EC2, Rust, TypeScript.

Microsoft
Principal Software Engineer **August, 2022 - September, 2022**
Senior Software Engineer **March, 2019 - August, 2022**
Software Engineer II **August, 2015 - March, 2019**

Tech lead for Linux in Microsoft's distributed build-acceleration service, and long-time contributor to BuildXL, the build engine used for Windows and Office. Built the low-level systems behind both: process sandboxing, process remoting, and filesystem virtualization. Ran team stand-ups, presented status at weekly execution reviews with leadership of Microsoft's One Engineering System (1ES) organization, and covered for the team lead during their leave.

Remote Build Acceleration for Linux. Built a two-week end-to-end prototype of remote build execution on Linux and demoed it to 1ES leadership, who green-lit the effort; the service became 1ES's primary build-acceleration technology for Linux. As tech lead for Linux, drove adoption end to end, from finding target teams to proofs of concept and onboarding, bringing on about eight internal teams (about 170 users in one half), e.g., cutting build times from 50 to 35 and from 40 to 25 minutes. Set technical direction: reworked cache fingerprints (fully cached LLVM build from 5 to 1 minute) and added per-command containerization to safely accelerate public repositories.

Filesystem Virtualization for Linux. Led the design, implementation, and optimization of a virtual filesystem for Linux that mirrors a client's filesystem on a remote server. The client's files are overlaid on the server's local filesystem and uploaded selectively, on demand, only when they are first needed. (FUSE, OverlayFS, POSIX, gRPC, C++, C#)

Process Sandboxing and Substitution for Linux. Led the design, implementation, and optimization of a library that observes a process's file accesses at run time and blocks unwanted ones. The library can also intercept selected child processes and run them remotely, inside a virtual filesystem on another machine. (LD_PRELOAD, rtld-audit, C++, C#, Azure VMs)

Process Sandboxing for macOS. One of two main contributors to a process sandboxing library implemented as a macOS kernel extension. Designed and implemented a concurrent lock-free trie and used it to add several layers of caching to the kernel-mode code, improving performance by about 35%. Brought fully sandboxed builds of Office for Mac from over 5 hours to about 3 hours 15 minutes, within 30% of an unsandboxed baseline, and cut kernel memory use from 8 GB to 1.5 GB; selected as one of 1ES's quarterly highlights. (Kext, KAuth, TrustedBSD, C++, C#)

Debugger and Query Language for Analyzing Build Logs. Designed and implemented a first-order relational query language for analyzing build logs, with results displayed graphically in a VS Code debugging session. (VS Code Extension; C#, TypeScript)

Daemon Process Support in the Build Engine. Build engines typically start a process only after all of its dependencies have finished, but some processes, such as services and daemons, need to run throughout the build, mostly for performance reasons. Rather than hard-coding a one-off solution for our immediate need, successfully argued for generic support in the engine. Designed the abstractions and implemented the original use case, reducing build tail time by 90%. Since adopted across Windows and Office builds. (C#)

Education

Massachusetts Institute of Technology, Cambridge, Massachusetts USA

Ph.D., Computer Science, May 2015
- Topic: "*Advancing Declarative Programming*" (advised by Prof. Daniel Jackson)

M.S., Computer Science, September 2010
- Topic: "*Executable Specifications for Java Programs*" (advised by Prof. Daniel Jackson)

School of Electrical Engineering, Belgrade, Serbia

B.Sc. in Computer Science, November 2007
- Topic: Parallel Test Generation and Execution with Korat (advised by Prof. Dragan Milicev)

Academic Experience

Massachusetts Institute of Technology, Cambridge, Massachusetts, USA

Research Assistant **August, 2008 - May 2015**
Includes Ph.D. research, Masters-level coursework and research projects.

Teaching Assistant **Spring 2009, Fall 2009**
"6.005 Elements of Software Construction": gave recitations, graded problem sets and projects.

Research Internships

Microsoft Research, Redmond, WA, USA
Research intern **June, 2011 - August, 2011**
Worked with Rustan Leino on program synthesis from first-order declarative specifications.

Microsoft Research Cambridge, Cambridge, United Kingdom
Research intern **June, 2009 - August, 2009**
Worked with Hillel Kugler on analyzing and executing Live Sequence Charts using SMT.

University of Illinois at Urbana Champaign, Urbana, Illinois, USA
Visiting Scholar **August, 2006 - September, 2006**
Worked with Darko Marinov on bounded-exhaustive test input generation.

Early Industry Experience

Serbian Object Laboratories, Belgrade, Serbia
Software Engineer **March, 2006 - August, 2008**
Actively worked on the development of the EDMT Server (www.bmmsoft.com). Technologies used: WebWork, Java Servlets, WS, SOAP, JSP, HTML, CSS, JS, AJAX, with Sybase IQ database.

Google Inc., New York, New York, USA
Software Engineering Intern **July, 2007 - September, 2007**
Worked with Nemanja Petrovic on decoding barcodes from images taken with a cell phone.

Selected Research Projects

- *Alloy** (`https://aleksandarmilicevic.github.io/hola`): a general-purpose, higher-order relational constraint solver (over bounded domains).
- *The Alloy Analyzer* (`https://alloytools.org`): an automated model finder for a first-order relational specification language.
- *Korat* (`http://korat.sourceforge.net`): a tool for bounded-exhaustive generation of test inputs based on complex constraints the inputs must satisfy.

Publications

P. Nie, A. Çelik, M. Coley, **A. Milicevic**, J. Bell, M. Gligoric. Debugging the Performance of Maven's Test Isolation: Experience Report.*ISSTA 2020, Los Angeles, USA*

B. Buhse, T. Wei, Z. Zang, **A. Milicevic**, M. Gligoric. VeDebug: Regression Debugging Tool for Java *ICSE Demo 2019, Montreal, Canada*.

A. Çelik, M. Vasic, **A. Milicevic**, M. Gligoric. Regression Test Selection Across JVM Boundaries. *FSE 2017, Paderborn, Germany*.

M. Vasic, Z. Parvez, **A. Milicevic**, M. Gligoric. File-level vs. Module-level Regression Test Selection for .NET. *FSE 2017 (Industry Track), Paderborn, Germany*.

E. Kang, **A. Milicevic**, D. Jackson. Multi-Representational Security Analysis, *FSE 2016, Seattle, WA, USA*. [Distinguished Paper Award]

A. Çelik, A. Knaust, **A. Milicevic**, M. Gligoric. Build System with Lazy Retrieval for Java Projects, *FSE 2016, Seattle, WA, USA.*

A. Milicevic. Advancing Declarative Programming, *Massachusetts Institute of Technology, Ph.D. Thesis, May 2015.*

A. Milicevic, J. P. Near, E. Kang, and D. Jackson. Alloy*: A Higher-Order Relational Constraint Solver, *ICSE 2015, Florence, Italy.* [Distinguished Paper Award]

A. Milicevic, J. P. Near, E. Kang, and D. Jackson. Alloy*: A Higher-Order Relational Constraint Solver, *MIT CSAIL Technical Report, September 2014.*

A. Milicevic, and D. Jackson. Preventing Arithmetic Overflows in Alloy (extended journal version), *Science of Computer Programming, May 2014.*

A. Milicevic, I. Efrati, and D. Jackson. αRby—An Embedding of Alloy in Ruby, *ABZ 2014, Toulouse, France.*

A. Milicevic, M. Gligoric, D. Marinov, and D. Jackson. Model-Based, Event-Driven Programming Paradigm for Interactive Web Applications, *Onward! 2013, Indianapolis, Indiana, USA*

K. R. M. Leino, and **A. Milicevic**. Program Extrapolation with Jennisys, *Splash 2012, Tucson, Arizona, USA.*

A. Milicevic, and D. Jackson. Preventing Arithmetic Overflows in Alloy, *ABZ 2012, Pisa, Italy.*

A. Milicevic, D. Rayside, K. Yessenov, and D. Jackson. Unifying Execution of Imperative and Declarative Code, *ICSE 2011, Waikiki, Honolulu, Hawaii.*

J. P. Near, **A. Milicevic**, E. Kang, D. Jackson. A Lightweight Approach to Construction and Evaluation of a Dependability Case, *ICSE 2011, Waikiki, Honolulu, Hawaii.*

A. Milicevic, and H. Kugler. Model Checking with SMT and Theory of Lists, *3rd NASA Formal Method Symposium (NFM 2011), Pasadena, California, USA.*

A. Milicevic. Executable Specifications for Java Programs, *Massachusetts Institute of Technology, Master's Thesis, September 2010.*

D. Rayside, **A. Milicevic**, K. Yessenov, G. Dennis, and D. Jackson. Agile Specifications, *OOPSLA Onward! 2009 (short paper), Orlando, Florida, USA.*

D. Rayside, Z. Benjamin, J. Near, R. Sing, **A. Milicevic**, and D. Jackson. Equality and Hashing for (almost) Free: Generating Implementations from Abstraction Functions, *ICSE 2009, Vancouver, Canada.*

S. Misailovic, **A. Milicevic**, N. Petrovic, S. Khurshid, and D. Marinov. Parallel Test Generation and Execution with Korat, *ESEC/FSE 2007, Dubrovnik, Croatia.*

A. Milicevic, S. Misailovic, D. Marinov, and S. Khurshid. Korat: A Tool for Generating Structurally Complex Test Inputs, *ICSE Demo 2007, Minneapolis, Minnesota, USA.*

S. Misailovic, **A. Milicevic**, S. Khurshid, and D. Marinov. Generating Test Inputs for Fault-Tree Analyzers using Imperative Predicates, *STEP 2007, Memphis, Tennessee, USA*